

Session 5

Model Evaluation & Metrics

How Do We Know If Our Model Is Good?

Why Model Evaluation Matters

The 99% Accuracy Trap

Scenario: You build a fraud detection model. It achieves 99% accuracy!

But wait... Only 1% of transactions are fraud.

A model that predicts "NOT FRAUD" for everything also gets 99% accuracy!

But it catches ZERO fraudsters. Completely useless!

KEY INSIGHT: Accuracy alone is NOT enough. We need multiple metrics to understand model performance, especially for imbalanced datasets.

The Confusion Matrix: Foundation of All Metrics

A 2x2 table that shows EVERYTHING about classification performance

		Predicted	
		Negative (0)	Positive (1)
Actual	Negative (0)	TN	FP
	Positive (1)	FN	TP

TN = True Negative

Correctly predicted negative

FP = False Positive (Type I Error)

Wrongly predicted positive
"False Alarm"

FN = False Negative (Type II Error)

Wrongly predicted negative
"Missed Detection"

TP = True Positive

Correctly predicted positive

💡 **MEMORY TRICK:** First letter = Was the prediction correct? (T/F). Second letter = What did we predict? (P/N)

Confusion Matrix: Medical Diagnosis

Example

Cancer Screening: 1000 patients tested, 100 actually have cancer

	Predicted: No Cancer	Predicted: Cancer
No Cancer	TN = 850	FP = 50
Has Cancer	FN = 10	TP = 90

Reading the Numbers:

✓ **TN = 850**: Healthy patients correctly told "no cancer"

⚠ **FP = 50**: Healthy patients wrongly told "cancer"
(Unnecessary stress, follow-up tests)

✗ **FN = 10**: Cancer patients wrongly told "no cancer"
(DANGEROUS! Missed diagnosis!)

✓ **TP = 90**: Cancer patients correctly detected

? CRITICAL QUESTION: Which is worse here — FP or FN? In cancer screening, FN is catastrophic (missed cancer). We'd rather have more FPs (extra tests) than miss a single cancer case.

Metric 1: Accuracy

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN}) = \text{Correct} / \text{Total}$$

What it measures: "Of ALL predictions, what fraction was correct?"

Example (Cancer): Accuracy = $(90 + 850) / 1000 = 94\%$

When to use ✓

- Balanced classes (50/50 split)
- Quick overall performance check
- When all errors are equally bad

When NOT to use X

- Imbalanced data
- When some errors are worse than others
- Medical diagnosis, fraud detection, rare events



THE ACCURACY PARADOX

Fraud Detection Dataset:

- 10,000 transactions
- 100 are fraud (1%)
- 9,900 are legitimate (99%)

Lazy Model: "Predict NOT FRAUD always"

Accuracy = $9,900 / 10,000 = 99\%$!

But it catches **ZERO** fraudsters!

Metric 2: Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

The Question: "Of all the times I predicted POSITIVE, how many were actually positive?"

Also called: Positive Predictive Value (PPV)

Example (Cancer): Precision = $90 / (90 + 50)$
= $90/140 = 64.3\%$

Interpretation: "When we say 'cancer', we're right 64% of the time"



PRIORITIZE PRECISION WHEN:

False Positives are COSTLY!

Spam Detection

FP = Important email marked as spam
(You miss a job offer!)

Criminal Justice

FP = Innocent person convicted
(Terrible injustice!)

Marketing Targeting

FP = Wasted ads on uninterested people
(Money down the drain!)

Metric 3: Recall (Sensitivity)

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

The Question: "Of all the ACTUAL positives, how many did I find?"

Also called: Sensitivity, True Positive Rate (TPR), Hit Rate

Example (Cancer): $\text{Recall} = 90 / (90 + 10) = 90/100 = \mathbf{90\%}$

Interpretation: "We find 90% of cancer patients"



PRIORITIZE RECALL WHEN:

False Negatives are DANGEROUS!

Cancer Screening

FN = Missed cancer diagnosis
(Patient dies!)

Fraud Detection

FN = Fraud goes undetected
(Money lost!)

Security Systems

FN = Intruder not detected
(Security breach!)

The Precision-Recall Tradeoff

You can't maximize both! There's always a tradeoff.

High Threshold (0.9)

"Only predict positive if VERY sure"

Precision: HIGH 
(Few FPs, very confident)

Recall: LOW 
(Miss many positives)

Balanced Threshold (0.5)

"Standard decision boundary"

Precision: MEDIUM ↔

Recall: MEDIUM ↔

(Balanced tradeoff)

Low Threshold (0.1)

"Predict positive if ANY doubt"

Precision: LOW 
(Many false alarms)

Recall: HIGH 
(Catch almost all positives)

As threshold ↑: Precision ↑, Recall ↓ | **As threshold ↓: Precision ↓, Recall ↑**

 The right tradeoff depends on your domain! Cancer → prioritize Recall. Spam → prioritize Precision.

Metric 4: F1-Score (Harmonic Mean)

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

What it does: Balances Precision and Recall into one number

Why HARMONIC mean (not regular average)?

- Regular average: $(0.9 + 0.1) / 2 = 0.5$ (seems OK)
- Harmonic mean: $2 \times (0.9 \times 0.1) / (0.9 + 0.1) = 0.18$ (reveals imbalance!)

The harmonic mean **penalizes extreme imbalances**. You can't cheat by making one very high and the other very low!

Example (Cancer):

Precision = 64.3%

Recall = 90%

$F1 = 2 \times (0.643 \times 0.90) / (0.643 + 0.90)$

$F1 = 2 \times 0.579 / 1.543$

F1 = **75.0%**

Interpretation: Single score balancing both metrics

Metric 5: Specificity (True Negative Rate)

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$$

The Question: "Of all ACTUAL negatives, how many did I correctly identify as negative?"

Also called: True Negative Rate (TNR), Selectivity

Example (Cancer): Specificity = $850 / (850 + 50) = 850/900 = 94.4\%$

Interpretation: "We correctly identify 94.4% of healthy patients"



METRIC RELATIONSHIPS:

Sensitivity = Recall = TPR

→ Focus on POSITIVES

Specificity = TNR

→ Focus on NEGATIVES

False Positive Rate = 1 - Specificity

→ Used in ROC curves!

Classification Metrics: Complete Summary

Metric	Formula	Question	Range
Accuracy	$(TP+TN) / \text{Total}$	Overall correct?	0 to 1
Precision	$TP / (TP+FP)$	Of predicted +, how many correct?	0 to 1
Recall	$TP / (TP+FN)$	Of actual +, how many found?	0 to 1
F1-Score	$2 \times P \times R / (P+R)$	Balance of P and R?	0 to 1
Specificity	$TN / (TN+FP)$	Of actual -, how many correct?	0 to 1

Quick Decision Guide: Which Metric to Use?

Balanced Classes?

→ Accuracy is OK

FN is dangerous?

→ Maximize Recall

FP is costly?

→ Maximize Precision

Be aware: F1 Ignores True Negatives!

F1, Precision, and Recall completely IGNORE True Negatives!

If correctly identifying negatives is important, use other metrics.

Which metrics use which cells?

Metric	TP	TN	FP	FN
Accuracy	✓	✓	✓	✓
Precision	✓	X	✓	X
Recall	✓	X	X	✓
F1-Score	✓	X	✓	✓
Specificity	X	✓	✓	X
Balanced Acc	✓	✓	✓	✓
MCC	✓	✓	✓	✓

✓ RECOMMENDATION:

For imbalanced binary classification:

1. MCC (uses all 4 cells)
2. Balanced Accuracy
3. F1 + Specificity together

Balanced Accuracy: Simple Fix for Imbalanced Data

$$\text{Balanced Accuracy} = (\text{Sensitivity} + \text{Specificity}) / 2 = (\text{TPR} + \text{TNR}) / 2$$

What it does:

Gives EQUAL weight to both classes, regardless of their size!

Regular Accuracy problem:

- 95% negative, 5% positive
- Predict all negative → 95% accuracy!
- But Sensitivity = 0% (caught nothing!)

Balanced Accuracy solution:

- Sensitivity = 0%, Specificity = 100%
- Balanced Acc = $(0\% + 100\%) / 2 = 50\%$
- Correctly shows the model is no better than random!

```
from sklearn.metrics import \
    balanced_accuracy_score

bal_acc = balanced_accuracy_score(
    y_test, y_pred
)
```



USE WHEN:

- Classes are imbalanced
- Both classes are equally important
- You want a simple, interpretable metric

MCC: Matthews Correlation Coefficient

The BEST single metric for imbalanced binary classification!

$$\text{MCC} = (\text{TP} \times \text{TN} - \text{FP} \times \text{FN}) / \sqrt{[(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})]}$$

Why MCC is special:

- ✓ **Uses ALL 4 cells** (TP, TN, FP, FN)
- ✓ **Balanced metric** (works with any class distribution)
- ✓ **Correlation interpretation** (like Pearson's r)

Range: -1 to +1

- MCC = +1: Perfect prediction
- MCC = 0: Random prediction
- MCC = -1: Total disagreement



MCC vs Other Metrics:

Scenario: 99% negative, 1% positive
Model: Always predicts negative

Accuracy = 99% (looks great!)
F1-Score = 0% (no positives found)
MCC = 0 (correctly shows random!)

→ **MCC reveals the model is useless!**

Threshold Optimization: Don't Use 0.5 Blindly!

The default threshold of 0.5 is often NOT optimal for your problem!

Youden's J Statistic

$J = \text{TPR} - \text{FPR}$
= Sensitivity + Specificity - 1

Maximize J to find optimal threshold

Use when: Balanced tradeoff

Maximize Recall

Lower the threshold!

More things predicted as positive → catch more actual positives

Use when: FN is dangerous (cancer, fraud)

Maximize Precision

Raise the threshold!

Only predict positive when very confident

Use when: FP is costly (spam, criminal justice)

```
# Find optimal threshold using Youden's J
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
J = tpr - fpr # Youden's J statistic
optimal_idx = np.argmax(J)
optimal_threshold = thresholds[optimal_idx] # Often NOT 0.5!
```

Multiclass Metrics: Macro vs Micro vs Weighted

When you have 3+ classes, how do you average Precision/Recall/F1?

MACRO Average

Calculate metric per class, then average equally.

$$F1_macro = (F1_A + F1_B + F1_C) / 3$$

Use when: All classes equally important (regardless of size)

MICRO Average

Aggregate TP, FP, FN globally, then calculate metric.

$$F1_micro = 2 * TP / (2 * TP + FP + FN)$$

Use when: Large datasets, class imbalance is OK

WEIGHTED

Calculate metric per class, then average weighted by class size.

$$F1_weighted = \sum (F1_i * n_i) / \sum n_i$$

Use when: Imbalanced datasets, larger classes matter more

Example: Sentiment Analysis (Positive: 100 samples, Neutral: 500, Negative: 400)

- **Macro:** Each sentiment weighted equally (good if all sentiments important)
- **Weighted:** Neutral & Negative dominate (reflects real distribution)

The ROC Curve: Visualizing All Thresholds

ROC = Receiver Operating Characteristic

What is it?

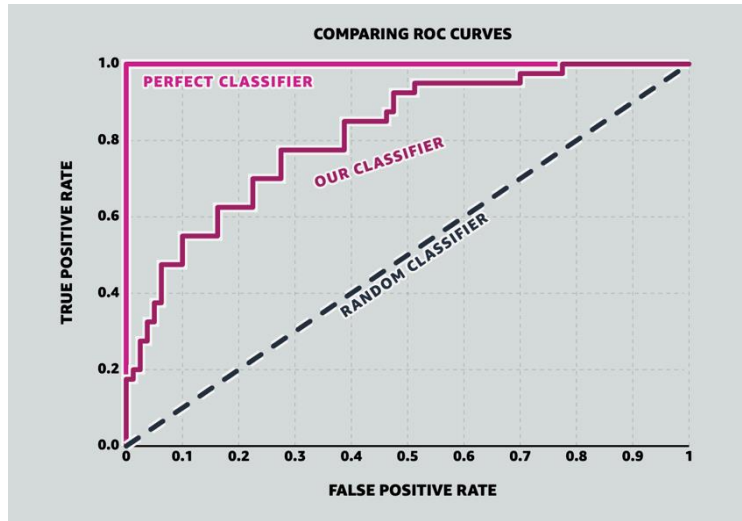
A curve that shows model performance at **ALL possible thresholds**

Axes:

- **X-axis:** False Positive Rate = $FP/(FP+TN) = 1 - \text{Specificity}$
- **Y-axis:** True Positive Rate = $TP/(TP+FN) = \text{Recall}$

How to read it:

- Each point = one threshold setting
- Curve shows tradeoff between TPR and FPR
- **Higher and more left = BETTER**



AUC: Area Under the ROC Curve

AUC = Area Under the ROC Curve (0 to 1)

What it measures:

"Probability that a random positive sample is ranked higher than a random negative sample"

Interpretation:

- **AUC = 0.5:** Random guessing (useless)
- **AUC = 0.7-0.8:** Acceptable
- **AUC = 0.8-0.9:** Good
- **AUC = 0.9-1.0:** Excellent
- **AUC = 1.0:** Perfect (suspicious! check for leakage)



WHY AUC IS POWERFUL:

1. Threshold-independent

Evaluates model across ALL thresholds

2. Scale-invariant

Works with probabilities, not raw scores

3. Class-imbalance robust

Less affected by imbalanced classes (but not immune!)

4. Easy to compare models

Single number: higher = better

Precision-Recall Curve (for Imbalanced Data)

When to use PR Curve over ROC?

When you have **highly imbalanced** data (rare positive class)

Why?

ROC can be misleadingly optimistic with imbalanced data.

PR curve focuses on the **positive class**, which is what we care about.

Axes:

- X-axis: Recall (TPR)
- Y-axis: Precision

ROC vs PR Curve:

ROC Curve:

- Good for balanced data
- X: FPR, Y: TPR
- Baseline: diagonal (AUC=0.5)
- Less sensitive to class imbalance

PR Curve:

- Best for imbalanced data
- X: Recall, Y: Precision
- Baseline: horizontal at prevalence
- Reveals poor performance on minority



Average Precision (AP): Area under PR curve. Summary metric like AUC but for PR curves.

Regression Metrics: MSE & RMSE

MSE (Mean Squared Error)

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

- Average of squared errors
- Units: squared (e.g., €²)
- Heavily penalizes large errors

Example: House Price Prediction

Predictions: [250k, 300k, 400k]

Actual: [260k, 280k, 420k]

Errors: [-10k, +20k, -20k]

$$\text{MSE} = (100 + 400 + 400) / 3 = \mathbf{300 \text{ million } €^2}$$

(Not very interpretable!)

$$\text{RMSE} = \sqrt{300\text{M}} = \mathbf{€17,320}$$

(Average error is about €17k - much clearer!)

RMSE (Root Mean Squared Error)

$$\text{RMSE} = \sqrt{\text{MSE}}$$

- Square root of MSE
- Units: same as target (e.g., €)
- More interpretable!

KEY INSIGHTS:

- RMSE is in original units (easier to explain to stakeholders)
- Both penalize large errors heavily (due to squaring)
- Sensitive to outliers (one big error dominates)

Regression Metrics: MAE

$$\text{MAE} = (1/n) \sum |y_i - \hat{y}_i|$$

MAE (Mean Absolute Error)

What it measures:

Average absolute difference between predicted and actual values

Properties:

- Units: same as target (like RMSE)
- Treats all errors equally (no squaring)
- **More robust to outliers!**

Example:

Errors: [-10k, +20k, -20k]

$\text{MAE} = (10 + 20 + 20) / 3 = \text{€}16,667$

MSE/RMSE vs MAE: When to Use

	RMSE
Outlier Sensitivity	HIGH (squares the errors)
Interpretation	Typical error size
Use when...	Large errors are bad
Example	Safety-critical systems

Regression Metrics: R^2 (Coefficient of Determination)

$$R^2 = 1 - (SS_{\text{res}} / SS_{\text{tot}})$$

where $SS_{\text{res}} = \sum (y_i - \hat{y}_i)^2$ and $SS_{\text{tot}} = \sum (y_i - \bar{y})^2$

What it measures:

"What proportion of variance in y is explained by the model?"

Interpretation:

- $R^2 = 0$: Model explains nothing (same as predicting mean)
- $R^2 = 0.5$: Model explains 50% of variance
- $R^2 = 1.0$: Perfect fit (explains all variance)
- $R^2 < 0$: Model is WORSE than just predicting the mean!

UNDERSTANDING R^2 :

SS_{tot} = Total variance in y
(How far are points from the mean?)

SS_{res} = Residual variance
(How far are points from predictions?)

$R^2 = 1 - (\text{unexplained} / \text{total})$
= proportion explained

If SS_{res} is small $\rightarrow R^2$ is high $\rightarrow$ good fit

Regression Metrics: Complete Summary

Metric	Formula	Units	Outlier Robust?	R-squared
MSE	$(1/n)\sum(y-\hat{y})^2$	Squared	No	0 to 1
RMSE	$\sqrt{\text{MSE}}$	Same as y	No	0 to 1
MAE	$(1/n)\sum y-\hat{y} $	Same as y	Yes	0 to 1
R ²	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Unitless	No	-∞ to 1

Quick Decision Guide:

Reporting to stakeholders?

→ Use RMSE or MAE
(same units as target)

Comparing model fit?

→ Use R²
(proportion explained)

Dataset has outliers?

→ Use MAE
(more robust)

The Bias-Variance Tradeoff

Every model's error can be decomposed into: $\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$

BIAS (Underfitting)

- Model is too **SIMPLE**
- Can't capture the true pattern
- High error on BOTH train and test
- Example: Fitting a line to curved data

VARIANCE (Overfitting)

- Model is too **COMPLEX**
- Memorizes training data (including noise)
- Low train error, HIGH test error
- Example: Fitting every single point exactly

Bullseye Analogy:

Low Bias, Low Var
✓ Ideal: Accurate & consistent

Low Bias, High Var
⚠ Right on average, but scattered

High Bias, Low Var
⚠ Consistent but wrong

High Bias, High Var
✗ Worst of both

How to Detect Overfitting & Underfitting

Scenario	Train Error	Test Error	Diagnosis	Solution
Underfitting	HIGH	HIGH	Too simple	More features, comp
Good Fit	LOW	LOW	✓ Ideal	Keep it!
Overfitting	LOW	HIGH	Too complex	Regularization, m
Data Problem	LOW	VERY HIGH	Leakage?	Check data pip



THE KEY DIAGNOSTIC: Compare training error vs validation/test error.

A big GAP between them = overfitting. Both high = underfitting. Both low = good fit!

Solutions:

Overfitting: More data, regularization (L1/L2), simpler model, dropout, early stopping

Underfitting: More features, complex model, less regularization, train longer

Sklearn Implementation

```
# Classification Metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve
)
```

```
# Regression Metrics
from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error,
    r2_score
)

# RMSE
rmse = mean_squared_error(
    y_true, y_pred,
    squared=False
)
```

```
# Quick evaluation
print(classification_report(y_test, y_pred)) # All metrics at once!

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
auc = roc_auc_score(y_test, y_proba)
plt.plot(fpr, tpr, label=f'AUC = {auc:.3f}')
```

Session 5 Summary

Classification Metrics:

- Confusion Matrix → Foundation (TN, FP, FN, TP)
- Accuracy → Overall correct (careful with imbalanced!)
- Precision → Of predicted +, how many correct?
- Recall → Of actual +, how many found?
- F1 → Balance of Precision & Recall
- ROC/AUC → Performance across all thresholds

Regression Metrics:

- MSE/RMSE → Average squared error (penalizes large errors)
- MAE → Average absolute error (robust to outliers)
- R^2 → Proportion of variance explained

Key Concepts:

- Bias-Variance Tradeoff → Under vs Overfitting
- Compare train vs test error to diagnose problems